OLLAMA LECTURE

SESSION 01 · LECTURE COVER

Ollama 완전 정복

로컬 AI 환경 구축부터 Python 채팅 앱까지

100%	5분	100+
무료·오프라인	설치 완료	지원 모델 수

PROGRAM
Ollama 로컬 LLM

SESSION
01 / 05

GOAL
설치 ~ 채팅앱 구현

FORMAT
강의 + 실습

SLIDES
12장

PREREQ
Python 기초

SESSION 01 · 핵심 메시지

데이터는 내 컴퓨터 안에서만 처리된다

02

클라우드 AI는 편리하지만, 기업의 내부 데이터·고객 정보·영업 비밀이 외부 서버로 전송됩니다. Ollama는 인터넷 없이 내 PC에서 LLM을 실행해 보안·비용·속도 세 가지를 동시에 해결합니다.

클라우드 AI

과금 + 데이터 외부 전송

Ollama

무료 + 완전 로컬 처리

OLLAMA LECTURE

SESSION 01 · CONCEPT

Ollama란 무엇인가?

MODEL DOWNLOAD 중앙 라이브러리

공식 라이브러리에서
모델 가중치를 한 줄
명령으로 다운로드

Docker pull과 동일한 개념

MODEL RUN GPU/CPU 가속

선택한 모델을 메모리에
로드하여 실행

NVIDIA GPU 자동 감지
및 가속 지원

REST API OpenAI 호환

localhost:11434 에서
HTTP API 제공

OpenAI 클라이언트
라이브러리와 호환

시스템 요구사항

RAM 8GB (7B 모델) / 16GB 이상 권장 | 저장 공간 10GB+ | Windows / macOS / Linux 모두 지원 | GPU 없이도 CPU 실행 가능

OLLAMA LECTURE

SESSION 02 · INSTALL

설치 및 환경 구성

WINDOWS

ollama.com

① ollama.com 접속
② Download for Windows 클릭
③ .exe 파일 실행 → 자동 설치
④ 작업표시줄에서 상태 확인

설치 완료 후 CMD 또는
Windows Terminal 에서 사용

MACOS

Ollama.app

① ollama.com에서 macOS 다운로드
② .zip 압축 해제 후 응용 프로그램 폴더로 이동
③ Ollama.app 실행

LINUX

```
# 터미널에 한 줄 입력으로 설치 완료
curl -fsSL https://ollama.com/install.sh | sh
```

설치 확인

```
ollama --version     # 버전 정보 출력되면 설치 성공
```

OLLAMA LECTURE

SESSION 02 · WINDOWS INSTALL

Windows PowerShell 설치 명령어

명령어

```
irm https://ollama.com/install.ps1 | iex
```

명령어 분해

irm

Invoke-RestMethod

URL에서 스크립트 내용을 다운로드

iex

Invoke-Expression

다운로드한 내용을 즉시 실행

KEY TAKEAWAY

Linux의 curl -fsSL ... | sh 와 완전히 동일한 패턴. PowerShell을 관리자 권한으로 실행한 뒤 한 줄만 입력하면 설치 완료.

OLLAMA LECTURE

SESSION 03 · COMMANDS

주요 명령어 완전 정복

명령어	기능	예시
`ollama pull <모델명>`	모델 다운로드 (실행 없이)	`ollama pull gemma4:e4b`
`ollama run <모델명>`	다운로드 + 즉시 실행	`ollama run gemma4:e4b`
`ollama list`	설치된 모델 목록 확인	`ollama list`
`ollama ps`	실행 중 모델 + 컨텍스트 길이	`ollama ps`
`ollama rm <모델명>`	모델 삭제	`ollama rm gemma4:e2b`
`ollama serve`	서버 수동 시작	`ollama serve`
`ollama show <모델명>`	모델 상세 정보 확인	`ollama show gemma4:26b`

KEY TAKEAWAY

pull = 다운로드만 / run = 다운로드 + 실행. 대용량 모델은 미리 pull 해두고 필요할 때 run 하는 것이 효율적입니다.

OLLAMA LECTURE

SESSION 03 · CHAT MODE

채팅 모드 내 특수 명령어

/bye 또는 Ctrl+D
채팅 종료

/clear
대화 기록 초기화

/set system "내용"
시스템 프롬프트 설정

/show info
현재 모델 정보 표시

실행 흐름 예시

```
# 모델 실행 (채팅 모드 진입)
$ ollama run gemma4:e4b

>>> 시스템 역할 지정
/set system "당신은 AX 전문 컨설턴트입니다."

>>> 질문 입력
디지털 전환의 핵심 성공 요소는?

>>> 대화 기록 초기화 후 종료
/clear
/bye
```

OLLAMA LECTURE

SESSION 04 · MODELS

모델 선택 기준

LOW SPEC RAM 8GB 이하

Phi-3 (3.8B)
Gemma4:E2B

DAILY USE

일반 업무 / 한국어

Gemma4:E4B ★
권장 입문 모델

REASONING 추론 / 수학 / 코딩

DeepSeek-R1:8b
Qwen3:8b

HIGH SPEC RAM 20GB+ 고성능

Gemma4:26b
Gemma4:31b

MODEL	파라미터	파일 크기	필요 RAM	특징
Phi-3	3.8B	~2.3GB	4GB	경량, 빠름
Llama3.2	3B	~2GB	4~6GB	범용성
Gemma4:E2B ★	2B (유효)	7.2GB	8GB	온디바이스, 멀티모달
Gemma4:E4B ★	4B (유효)	9.6GB	10GB	범용 권장, latest 태그
Qwen3:8b	8B	~5GB	10~12GB	한국어·추론·29개 언어
DeepSeek-R1:8b	8B	~5GB	10~12GB	추론·코딩 특화
Gemma4:26b	26B MoE(활성4B)	18GB	20GB+	고성능, 4B 연산 비용

SESSION 04 · GEMMA 4 SPOTLIGHT

Gemma 4 — 2026년 4월 Google DeepMind 공개

E2B

스마트폰·저사양
PC용 온디바이스

7.2GB

E4B ★

범용 일상 모델
latest 태그

9.6GB

26B MoE

4B 연산비용으로
26B 성능

18GB

31B Dense

서버급 최고 성능
고사양 전용

20GB

Apache 2.0 라이선스 — 상업적 활용·파인튜닝·SaaS 구축 모두 무료
텍스트 + 이미지 멀티모달 / 256K 컨텍스트 / 140개 이상 언어 지원

```
ollama pull gemma4        # latest = E4B
```

OLLAMA LECTURE

SESSION 04 · HANDS-ON

추천 모델 다운로드

Gemma4 (Google) ★ 권장

```
# 입문자 권장 — latest 태그
ollama pull gemma4:e4b

# 저사양 PC용
ollama pull gemma4:e2b

# 고성능 (RAM 20GB+)
ollama pull gemma4:26b
```

Qwen3 (Alibaba) — 한국어·추론

```
# 8B 추천
ollama pull qwen3:8b

# 고성능
ollama pull qwen3:32b
```

DeepSeek-R1 — 추론·코딩

Llama 3.2 (Meta) — 범용 경량

KEY TAKEAWAY

ollama run 명령어는 모델이 없으면 자동 다운로드 후 실행합니다. 미리 pull 해두면 실행 시 즉시 시작됩니다.

OLLAMA LECTURE

SESSION 05 · PYTHON CHAT

Python으로 채팅 앱 만들기

STEP 1 — 환경 준비

```
pip install ollama
python --version   # 3.8 이상 필요
```

STEP 2 — 단일 응답 (가장 간단한 형태)

```
import ollama
res = ollama.generate(model="gemma4:e4b", prompt="AI란?")
print(res["response"])
```

STEP 3 — 대화 기록이 유지되는 채팅 (핵심 패턴)

```
import ollama
messages = []                                   # 대화 기록 저장
while True:
    user_input = input("나: ")
    if user_input.lower() == "exit": break
    messages.append({"role": "user", "content": user_input})
    res = ollama.chat(model="gemma4:e4b", messages=messages)
    reply = res["message"]["content"]
    messages.append({"role": "assistant", "content": reply})
    print(f"AI: {reply}")
```

OLLAMA LECTURE

SESSION 05 · ADVANCED

스트리밍 & 역할 부여

스트리밍 출력 (실시간 타이핑 효과)

```
import ollama

messages = []
user_input = input("나: ")
messages.append({"role":"user","content":user_input})

print('AI: ', end='', flush=True)
full = ''
for chunk in ollama.chat(
    model="gemma4:e4b",
    messages=messages, stream=True):
    token = chunk["message"]["content"]
    print(token, end='', flush=True)
```

시스템 프롬프트로 역할 부여

```
import ollama

# 역할 지정 (시스템 메시지 먼저)
messages = [{
    "role": "system",
    "content": "당신은 AX 전문 컨설턴트입니다.
    모든 답변은 비즈니스 관점에서
    간결하게 제공하세요."
}]

messages.append({"role":"user",
"content":"DX 성공 요소는?"})
```

KEY TAKEAWAY

stream=True → 토큰 단위로 실시간 출력 (ChatGPT 타이핑 효과). 시스템 프롬프트로 AI 역할을 지정하면 일관된 전문가 응답을 얻을 수 있습니다.

SESSION 05 · WRAP-UP

자주 묻는 문제 & 마무리

증상	원인	해결
connection refused 오류	Ollama 서버 미실행	ollama serve 실행 또는 앱 재시작
응답이 너무 느림	CPU만 사용 중	GPU 드라이버 확인 / 작은 모델 변경
저장공간 부족	모델 용량 누적	ollama rm <모델> 로 미사용 삭제
한국어 품질 불량	모델 학습 부족	Gemma4:e4b 또는 Qwen3:8b 로 변경
model not found	모델명 오류	ollama list 로 정확한 이름 확인

다음 단계 추천:
Open WebUI 설치 → Modelfile 커스터마이징 → LangChain 연동 → RAG 구축